Eunomia

Common Errors Daily Crash Course

03/06:

Missing Coverage & Golden Response

Outlier

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

QC Fail #1: Missing Coverage

Each explicit requirement in the prompt must be checked by a rubric!!

🚨 For multi-turn tasks, you must account for requirements across the entire conversation history, not just the final turn.

Example of a Recent QC Fail:

- Conversation History:
- Turn 1: "Write a function..." -> Model Response: ....
- Last user prompt: "in short"
- The Error: The CB wrote rubrics for the function logic but failed to include a rubric for brevity/minimal code.

QC Fail Reason: *[Fail - Missing Coverage] No criterion enforces overall brevity despite the prompt explicitly requesting "in short."*

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

QC Fail #1: Missing Coverage

💡 **How to Avoid This Error: Pay more attention to details!**

1. Read all turns. Each explicit requirement (e.g., "use Python," "add comments," "make it concise," "no libraries") must have a corresponding rubric.
2. Apply "Stacking" Constraints: Unless they conflict, instructions from previous turns carry over. If Turn 1 asks for React and Turn 3 asks for a button, your rubric set must check for both React and the button. (Also make sure they are atomic (one rubric should only check a single thing at a time)
3. Even short phrases like "in short," "no comments in the code," at the end of a prompt are explicit requirements.

⚖️ **Handling Edge Cases**

When building rubrics for complex histories, follow these priority rules:

- Conflict Resolution: If the most recent prompt contradicts an earlier instruction, follow the most recent instruction.
- Unrelated Context: If the conversation shifts to a completely different topic, focus only on **the most recent prompt's** requirements.

Ambiguous Prompts: If a prompt is poorly written or confusing, choose non-ratable!

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

QC Fail #1: Missing Coverage Example

QC feedback:

[Fail - Missing Coverage] "The rubric is missing at least one criteria that checks for an explicit requirement in the prompt (only flag explicitly written requirement here) . c1-c6 ensure CRUD correctness; c7 limit prose; c8-c12 cover OoL. → No criterion enforces overall brevity or minimal code **despite prompt explicitly requested "in short".** → A response could pass all criteria yet remain length and not truly concise

User

write a c# program to demonstrate complete crud operations using asp.net

Assistant

Here's a **concise example** of a C# ASP.NET Web Forms program demonstrating complete **CRUD (Create, Read, Update, Delete)** operations with a SQL Server database. I'll keep it straightforward for understanding.

Step 1: Database Setup

Create a database `Bank` and a table `Customer`:

SQL

```sql
CREATE TABLE Customer (
    AccountNo INT PRIMARY KEY,
    Name NVARCHAR(50),
    Address NVARCHAR(100),
    Balance DECIMAL(18,2)
);
```

QC Fail #1: Missing Coverage

QC feedback:

[Fail - Missing Coverage] "The rubric is missing at least one criteria that checks for an explicit requirement in the prompt (only flag explicitly written requirement here) . c1-c6 ensure CRUD correctness; c7 limit prose; c8-c12 cover OoL. → No criterion enforces overall brevity or minimal code **despite prompt explicitly requested "in short".** → A response could pass all criteria yet remain length and not truly concise

```
14      protected void btnInsert_Click(object sender, EventArgs e)
15      {
16          using (SqlConnection con = new SqlConnection(connectionString))
17          {
18              string query = "INSERT INTO Customer (AccountNo, Name, Address, Balance) VALUES (@AccountNo, @Name, @Address,
                @Balance)";
19              SqlCommand cmd = new SqlCommand(query,
20              cmd.Parameters.AddWithValue("@AccountN                                    ext);
```

See more

✅ **Features:**

1. Insert, Update, Delete, and View operations.
2. Uses `SqlConnection`, `SqlCommand`, and `SqlDataAdapter`.
3. Displays data in `GridView`.

If you want, I can also **make a shorter version using ASP.NET MVC with Entity Framework**, which is cleaner and more modern than Web Forms. It's better for real-world applications.

Do you want me to do that?

Below is the Prompt that will be used to feed the 4 models. Read the prompt and check the ratability of the entire conversation.

Prompt

in short

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

QC Fail #1: Missing Coverage

1 **The code is written in C#.**
Code Rubric · Mandatory

2 **The code uses the ASP.NET framework.**
Code Rubric · Mandatory

3 **The response includes code for creating a record.**
Code Rubric · Mandatory

4 **The response includes code for reading records.**
Code Rubric · Mandatory

5 **The response includes code for updating a record.**
Code Rubric · Mandatory

6 **The response includes code for deleting a record.**
Code Rubric · Mandatory

7 **The response limits non-code content to section headers and single-sentence descriptions.**
Instruction Following · Mandatory

8 **The delete operation identifies records using a unique identifier such as a primary key.**
Quality Of Life-General Helpfulness · Nice-To-Have

9 **The response labels each CRUD operation with explicit comments or dedicated section headers.**
Quality Of Life-General Helpfulness · Nice-To-Have

10 **The response avoids including setup instructions such as migration commands or package installations.**
Quality Of Life-Verbosity · Nice-To-Have

11 **The code example demonstrates CRUD operations without including any database connection strings, configuration files, or specific database providers.**
Quality Of Life-General Helpfulness · Nice-To-Have

12 **The response does not include offers to generate additional code versions.**
Quality Of Life-Verbosity · Nice-To-Have

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

QC Fail #1: Missing Coverage

QC feedback:

[Fail - Missing Coverage] "The rubric is missing at least one criteria that checks for an explicit requirement in the prompt (only flag explicitly written requirement here) . c1-c6 ensure CRUD correctness; c7 limit prose; c8-c12 cover OoL. → No criterion enforces overall brevity or minimal code **despite prompt explicitly requested "in short".** → A response could pass all criteria yet remain length and not truly concise

Each explicit requirement in the prompt must be checked by a rubric!!

The Mandatory IF rubrics generator actually provided a specific check for the explicit ask for “in short”:

Response B

Instruction Following (IF) Rubrics:

- **Criterion**: To satisfy the user's "in short" request, the response presents the core CRUD implementation in a single compact code block (one file) rather than spread across multiple separate files or long multi-file tutorials.
- **Criterion**: For "complete crud operations" the response includes both a read-all/list endpoint/action and a read-by-id (single-record) endpoint/action in the ASP.NET C# code.
- **Criterion**: Because the user asked for ASP.NET and asked for brevity ("in short"), the response uses an ASP.NET approach that can run without scaffolding view files (i.e., Minimal API or Web API endpoints) rather than requiring full MVC view scaffolding or Web Forms pages.
- **Criterion**: The response targets records for Update/Delete by a numeric primary key (Id or AccountNo) in the ASP.NET C# code, not by a non-unique field (e.g., Name).

Watch out for overly-prescriptive rubrics! Dont limit the response to a single implementation method.

Tip: always write rubrics giving examples using “e.g., Like, ”

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

QC Fail #2: Golden Response (GR) Rewrite

The rewritten GR must pass ALL rubrics you created!

QC feedback:

[Fail - Code Rewrite Requirement] "The rewrite does NOT pass at least one of the criteria" → the golden response fails C7 (multi-sentence Run/Test)→ it include multi-sentence explanation and a detailed " How to Run & Test" section instead of limiting non-code content to headers and single-sentence description.

Here is the shortest, most modern way to demonstrate CRUD using **ASP.NET Core Web API** with an **In-Memory Database** (so you don't even need SQL Server installed to run it).

1. Create a `Customer` Model

Plaintext

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

2. Create the Controller (The Logic)

This handles all CRUD operations: `GET`, `POST`, `PUT`, `DELETE`.

Plaintext

```
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.Linq;
```


CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

QC Fail #2: Golden Response (GR) Rewrite

The rewritten GR must pass ALL rubrics you created!

QC feedback:

[Fail - Code Rewrite Requirement] "The rewrite does NOT pass at least one of the criteria" → the golden response fails C7 (multi-sentence Run/Test)→ it include multi-sentence explanation and a detailed " How to Run & Test" section instead of limiting non-code content to headers and single-sentence description.

7 **The response limits non-code content to section headers and single-sentence descriptions.**

Instruction Following · Mandatory

GR fails C7 on this:

```
        if (customer == null) return NotFound();

        customers.Remove(customer);
        return NoContent();
    }
}
```

How to Run & Test

1. Paste this into an **ASP.NET Core Web API** project.
2. Run the project (F5).
3. Use the built-in **Swagger UI** (which opens automatically) to test the buttons:
   - **POST** to add data.
   - **GET** to see it.
   - **PUT** to change it.
   - **DELETE** to remove it.

Go back to check the scoring sheet of your 4 model responses, make sure you rewrite it until the GR can pass all rubrics! Also it must PASS all evals! (but evals can missflag / overflag)

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

QC Fail #2: Golden Response (GR) Rewrite

Use the LLM judge to help you find what to fix in your GR! Go back and fix to make GR perfect!!

This is the LLM judge for Res C (the cb chose Res C as the base to rewrite GR), and it actually flag FAIL for C7:
If you read this, you can catch extra fail that you may miss when you were first grading the model responses by yourself!

Tip: **Carefully Read the LLM judge** in later step, especially the one for the model that you selected to be the base of your GR rewrite!! It can help flag fails for your selected model, and thus it points out what you should fix to make GR perfect!

Rubric Criterion	Category	Weight	(1/0)	Decision	Justification
The code is written in C#.	code rubric	mandatory	1	PASS	All code snippets are in C#.
The code uses the ASP.NET framework.	code rubric	mandatory	1	PASS	Uses `Microsoft.AspNetCore.Mvc` and ASP.NET Core Web API conventions.
The response includes code for creating a record.	code rubric	mandatory	1	PASS	Contains a `POST` action that adds a `Customer`.
The response includes code for reading records.	code rubric	mandatory	1	PASS	Contains `GET` actions for all records and by ID.
The response includes code for updating a record.	code rubric	mandatory	1	PASS	Contains a `PUT` action that updates a `Customer`.
The response includes code for deleting a record.	code rubric	mandatory	1	PASS	Contains a `DELETE` action that removes a `Customer`.
The response limits non-code content to section headers and single-sentence descriptions.	instruction following	mandatory	0	FAIL	Includes multi-sentence introductory paragraph and a multi-step "How to Run & Test" list, exceeding the allowed single-sentence descriptions.
The delete operation identifies records using a unique identifier such as a primary key.	quality of life-general helpfulness	nice-to-have	1	PASS	`DELETE` uses the `id` parameter (primary key).
The response labels each CRUD operation with explicit comments or dedicated section headers.	quality of life-general helpfulness	nice-to-have	1	PASS	Comments like `// CREATE`, `// READ`, `// UPDATE`, `// DELETE` are present.
The response avoids including setup instructions such as migration commands or package installations.	quality of life-verbosity	nice-to-have	1	PASS	No migration or package-installation commands are given.
The code example demonstrates CRUD operations without including any database connection strings, configuration files, or specific database providers.	quality of life-general helpfulness	nice-to-have	1	PASS	Uses an in-memory static list; no connection strings or provider specifics.
The response does not include offers to generate additional code versions.	quality of life-verbosity	nice-to-have	1	PASS	No offers for alternative code versions are present.

Final Summary

The model response failed 1 times on the **Mandatory** rubric criteria (ignoring nice-to-have, ignored, or negated criteria).

Failed rubric count: 1

Eunomia Common Errors

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Thank You

Outlier

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Reviewer Checklist

See the Reviewer Checklist in the instruction viewer for an overview of how tasks are scored.

Rating	Action	Criteria (Checklist)
1 – Terrible	Critical Errors	🚨 Ratability Fail: Marked ratable but is non-English, lacks context (missing files/history), niche platform, or is a general question unrelated to code. 🚨 Effort Fail: Attempter put little to no effort into rubrics or justifications. 🚨 Golden Response Fail: The rewrite is broken, contains syntax errors or code bug, or does NOT pass All the criteria.
2 – Bad	Major Errors	🟠 Counterproductive Rubrics: At least one criterion penalizes good responses / rewards bad responses such that the inclusion of this criteria would make the response objectively worse. 🟠 Overly Prescriptive Rubrics: At least one criterion is overly prescriptive, that it fails technically sound, industry-standard solutions. 🟠 Redundancy: Two or more rubrics penalize the exact same error. 🟠 Golden Response: Rewrite fails to pass even one of the Mandatory rubrics created by the attempter. 🟠 Rubric Accuracy: Criterion contains a factual error or contradicts the prompt. 🟠 Missing Coverage: The rubric is missing at least one criteria that checks for an **explicit** requirement in the prompt (only flag explicitly written requirements here)

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Reviewer Checklist

See the Reviewer Checklist in the instruction viewer for an overview of how tasks are scored.

Rating	Action	Criteria (Checklist)
3 – Okay	**Noticeable Issues**	🟡 Scoring Rule Violation: Failed 1 Code/IF rubric or 30-50% of QoL rubrics (should be a Score 3). 🟡 Don't spend too much time on editing those Nice to Have QoL, and spend more time on writing **mandatory Instruction Following (IF) rubrics** that can differentiate (Fail) models. 🟡 Don't fail an attempt if there is not enough QoL (less than 40%), give it a 3 is fine. 🟡 QoL Rubrics: QoL rubric has All Pass for the 4 models, failed to differentiate model responses enough. (don't fail such attempts, but score it a 3) 🟡 Vague/Generic Rubrics: Non-QoL rubrics lack "Technical Teeth" (e.g., "Code is readable"). 🟡 Reiteration: Rubric simply repeats the prompt without defining a success standard. 🟡 Justification: The explanation is generic, one-sentence, or doesn't reference specific rubric failures.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Reviewer Checklist

See the Reviewer Checklist in the instruction viewer for an overview of how tasks are scored.

Rating	Action	Criteria (Checklist)
4 – Good	Minor Issues Only	🟢 Formatting/Tone: Minor issues in the rewrite or justification (e.g., conversational "fluff"). 🟢 Rubric Nuance: 0-20% of weights (Mandatory vs. Nice-to-Have) are slightly off but don't flip the overall score. 🟢 QoL Failures: Response failed <30% of QoL rubrics (should be a Score 4).
5 – Excellent	No Issues	✅ Perfect Task: Zero errors. Rubrics are atomic, self-contained, and provide the ground truth. ✅ MECE: Rubrics are Mutually Exclusive and Collectively Exhaustive (~10 total). ✅ Technical Teeth: Rubrics use "LIKE/e.g." to define valid technical paths.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Finding the Balance: Prescriptive vs. Generic

🛑 1. Avoid: Overly Prescriptive (Rigid) Rubrics

The Test: If a model provides a different but technically sound solution (different library/pattern), would this rubric force a failure?

- **Why it's bad:** It penalizes "correct" models for not being mind-readers.
- **Exceptions:** Only be prescriptive if the prompt explicitly demands a method (e.g., "Use a `for` loop") or for security (e.g., "Use parameterized queries").

☁️ 2. Avoid: Overly Vague (Generic) Rubrics

The Test: If you swapped this Python prompt for a Java prompt, would the rubric still work?

- **Why it's bad:** "Code is clean" or "Provides a summary" are filler. They don't evaluate the model's coding logic.

💡 The "Technical Teeth" Strategy

- **Use "LIKE / e.g.":** Give the judge a standard while remaining flexible.
  - *Bad:* "Uses initializer injection."
  - *Good:* "Uses a standard form of Dependency Injection (e.g., Initializer, Property, or Method injection)."
- **Provide Ground Truth:** Since the LLM Judge doesn't "know" the answer, you must define it.
  - *Bad:* "Decodes the file correctly."
  - *Good:* "Identifies the compression algorithm (LZ-String) and method (decompressFromBase64) to deserialize the format."

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Common Reviewer Checkpoints

Feature	❌ Bad (Failing)	✅ Better (Passing)
Golden Response	Has syntax errors, or Fails any of the rubrics or includes "fluff" like "I'd be happy to help!"	• **Rewrite the golden response until it passes ALL rubrics.** • Golden Responses must be professional and direct. Delete "filler" praise such as "Great question!", "Great use case!", or "I'd be happy to help!". • Contextual Accuracy: Avoid tone-deaf affirmations. For example, if a user shows an error, the response should not say "Exactly!". You should remove those!
Atomicity	"The code is efficient AND has comments."	Split into two rubrics.
Overly Prescriptive	"The code uses a for loop to iterate through the array." (Forces one method)	"The code iterates through the array using an **efficient method (e.g., for loop, map, or filter)**."
Self-Contained	"Response identifies the first president."	"Response identifies the first president as George Washington."
Coverage	Prompt asks for a docstring but no rubric checks for it.	Create a Mandatory rubric for every **explicit** prompt constraint.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Common Reviewer Checkpoints

Feature	❌ Bad (Failing)	✅ Better (Passing)
Redundancy	R1: "Proper imports." R2: "No unimported modules."	Combine into one: "All required modules (e.g., pandas) are imported."
Ratability	Guessing what the user means by "the template" when no file is attached.	Mark as **Unratable** due to missing context.
Technical Teeth / IF Depth	Spending high effort on generic QoL rubrics (e.g., "formatting") for simple, easy prompts.	Adding **Instruction Following (IF) constraints** to the prompt to force mandatory failures and increase technical difficulty.
Rubric Scaling & Complexity	Including an excessive number of rubrics for low-complexity prompts (e.g. a 13 rubric set for a prompt like "What is 1<<n in cpp")	If the prompt is very easy, add Instruction Following (IF) constraints. Remove unnecessary rubrics, crafting a set that completely covers the prompt with Mutually Exclusive and Collectively Exhaustive criteria.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Ratability

Failing Issue

Niche Prompts

- Niche prompts should be **non-ratable**.
- Do not rate prompts focused on a **specific product** or a **niche platform.**

Examples

- ✕ “How can I get the Facebook ads data?” (this was in the context of a chat involving collecting ads from various platforms)
- ✕ “Write an Apex trigger in Salesforce that updates the custom field *Status_Flag__c* on the Opportunity object whenever a related *Custom_Invoice__c* record is approved”
- ✕ How do I configure a custom Workday calculated field to extract a worker's third-level manager's cost center, factoring in matrix organization overrides?

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Golden Response

Failing Issue

[Fail - Code Rewrite Requirements]

The rewrite does NOT pass all the criteria.

If a rewrite fails any rubric, the task fails and the highest quality score it can receive is 2.

Example

- ✕ A rewritten response **fails to resolve** one or more rubric criteria issues that the original response failed.
- ✕ A rewritten response **introduces a new bug** leading to failure of any rubric criteria failed by the original model response.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Counterproductive Criteria

Failing Issue

Counterproductive Criteria

[Fail - Counterproductive Criteria]
At least one criterion penalizes good responses / rewards bad responses such that the inclusion of this criteria would make the response objectively worse.

Example

- ✕ A rubric imposes a technical constraint (like specific text-based class names) that is **incompatible** with the subject's fundamental architecture (e.g., randomized/hashed builds), effectively forcing a brittle solution.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Coverage & Accuracy

Failing Issue

[Fail - Criteria Accuracy]

- Checks for something that contradicts the prompt.
- Contains factual errors or misleading points.
- Not an explicit requirement and doesn't improve the response.

Examples

- ✕ **Misleading and/or factually incorrect** statements like *"The response implements a sorting algorithm that runs in O(nlogn), such as selection sort"*
- ✕ The prompt specifies Python, but the rubric requires JavaScript instead.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Rubrics Criteria Objectivity/Specificity

Failing Issue

Objectivity / Specificity

[Fail - Objectivity / Specificity]: Criterion corresponding to explicit requirements is subjective, **vague**, or unmeasurable.

Examples

❌	"The response should have good formatting"
✅	"The response includes Python code that adheres to PEP 8 indentation standards"
❌	"Code must be optimal"
✅	"The response uses a hash map to achieve O(n) time complexity for the lookup operation, avoiding nested loops."

Write criteria that encompass all requirements needed to fulfill this prompt. 15/15 completed

1 **For allow_multiple: true with a groups mapping, the script selects at most one item per group.**
Code Rubric · Mandatory

2 **The code must check that every group-selected item must be present in that category's items list.**
Code Rubric · Mandatory

3 **After group picks, the script fills remaining slots (up to max_picks) from items not already chosen.**
Code Rubric · Mandatory

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Rubrics Criteria Redundancy

Failing Issue

Overlap / Redundancy

[Fail - Overlap / Redundancy]: At least one criterion is completely redundant.

This double penalizes models and skews ratings.

Redundant Scenario 1: Criteria 1: Response does a, b, c Criteria 2: Response does a, b	**Redundant Scenario 2:** Criteria 1: Response does a, b, c Criteria 2: Response does b, c, d

Examples

Redundant Criteria:

- ❌ "The database schema definition includes an index on the email column of the users table."
- ❌ "The SQL migration script optimizes the users table for frequent lookups by email address."

Non-Redundant Criterion:

- ✅ "The database schema creation script defines a lookup structure on the users.email column (e.g. via a PRIMARY KEY, UNIQUE constraint, or CREATE INDEX statement).

Note: This applies to cases where two criteria independently assess the same elements, not when a single criterion introduces and specifies related requirements."The response follows best code practices by ensuring that each line is under 79 characters" is acceptable).

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Avoid Generic Rubrics

🛑 Error Type: Generic Rubrics (Boilerplate Criteria)

Definition: Using "copy-paste" criteria that are not tailored to the specific technical task. These are abstract placeholders (like asking for a summary, next steps, or specific formatting) that could apply to almost any prompt regardless of the programming language or problem.

The "Specific to Task" Test: Ask yourself: "If I swapped this Python prompt for a Java or SQL prompt, would this rubric still work perfectly?" If the answer is yes, the rubric is likely too generic and lacks "technical teeth."

Contextual Exceptions (When it's a Non-Fail):

- **Quality of Life (QoL) Threshold:** Generic criteria are acceptable **only** if they are classified as "Quality of Life" and weighted as **"Nice-to-Have."**

Example Case:

- **Prompt:** "How do I split a string in Python using a delimiter?"
- ❌ **Bad Practice (Generic/Boilerplate):**
  - **Criterion 8:** "The response offers follow-up questions or next steps."
  - **Criterion 11:** "The response provides a brief summary or recap."
  - **Criterion 12:** "The response avoids decorative or excessive formatting."
  - **Why it's weak:** These tell us nothing about Python string methods (`.split()`) or delimiters. They are "filler" rubrics.
- ✅ **Good Practice (Task-Specific):**
  - **Criterion 1:** "The response demonstrates the `.split()` method and explains how to pass a specific string delimiter as an argument."
  - **Criterion 2:** "The code correctly handles edge cases, such as splitting a string that does not contain the specified delimiter."
  - **Criterion 3 (Specific QoL):** "The response suggests using `re.split()` if the user needs to split by multiple different characters at once." (This is a "next step" that is actually **specific** to the task).

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Watch out for vague wording

Error Category	Issue Description	Bad Example (Vague/Dependent)	Good Example (Specific/Self-Contained)
Vague Adjectives	Using subjective words like "appropriate," "correctly," or "valid" which an LLM judge cannot define.	"The response suggests a tech stack that is appropriate for the scale."	"The response suggests a tech stack **using SQLite and a single Python script** for a scale of <10k records."
Lack of Ground Truth	Failing to specify the actual technical method required, forcing the judge to guess the "right" way.	"The response provides a technically valid alternative to track.label."	"The response provides an alternative **such as using the videoKind property** to distinguish the stream."
Non-Self-Contained	Requiring the judge to refer back to the prompt or external docs to understand the criteria.	"The model follows the specific instructions mentioned in the prompt."	"The model implements the AES-256 encryption using the **cryptography library as requested.**"
Placeholder Language	Using "filler" phrases that sound professional but lack technical "teeth."	"The response follows industry best practices for error handling."	"The response includes a **try-except block** that catches the specific FileNotFoundError."
Circular Logic	Defining the rubric by the success of the outcome rather than the steps to get there.	"The code works correctly and produces the right output."	"The code uses a list comprehension to filter the data, ensuring the output list contains only integers."
High-Weight Cosmetics	Giving too much importance to formatting (Quality of Life) instead of technical logic.	"The response uses perfectly formatted headers and bold text."	"The response uses Markdown headers to separate the 'Setup' and 'Execution' phases for readability."
QoL rubrics passing All 4 models	The project attempter instructions explicitly say that "Quality-of-Life rubric evaluates the differences between the responses. **A good QoL criterion cannot be passed by all four responses."** QoL rubrics MUST evaluate the differences between responses.	a QoL rubric that all 4 responses pass or all 4 fail.	QoL rubrics MUST evaluate the differences between responses. Check: Ask yourself: "Does this rubric actually help distinguish which model is better?" 🔍

02/13 TLDR: Stop using adjectives or vague words, start using technical nouns and give concrete examples for specific methods. 🎯

Example: Don't say "appropriate tech stack"; say "uses SQLite and Python." ✅
Don't say "uses a valid method"; say "uses the videoKind property." ✅
Don't say "explains clearly"; say "identifies the LZ-String algorithm." ✅

But also watch out for being too prescrptive, you should give multiple examples if the model uses different ways to implement a code, don't limited the model to a single method!!

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Self-contained Criteria

Failing Issue

Self-contained Rubric

[Fail - Self-contained Criteria]: At least one rubric criterion cannot be evaluated without access to prompt, reference text, or external facts.

Not self-contained

"Response identifies the first president of the USA"

"The response addresses the bug mentioned in the prompt

Self-contained

"Response identifies the first president of the USA as George Washington"

"The response addresses the bug where the submit button doesn't work"

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Rubrics should be self-contained!!

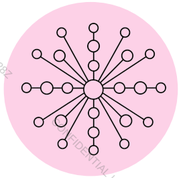
The Rubric **needs to be structured in a very specific way**, or the model might learn the wrong things from the rubric.

We can codify the features of a useful rubric as a set of principles.

Rubrics are highly prompt-specific, but...

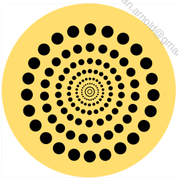
...the goal is a set of criteria that are **mutually exclusive** (they shouldn't overlap) and **collectively exhaustive** (taken together, they should fully define the ideal response).

MECE can be achieved by following **three principles:**



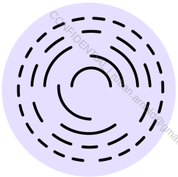
Atomicity

Each criterion should cover **exactly one** aspect



Specificity

Criteria should **precisely** state what is expected



Self-containment

Each criterion should contain **all the information** needed to evaluate a response.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Coverage

Failing Issue

Missing Coverage

[Fail - Missing Coverage]: The rubric is missing at least one criterion that checks for an explicit requirement in the prompt or a critical implicit expectation.

Example

The user request requires a Cloudflare Worker with a Cron Trigger that runs every 5 minutes, but the 5-minute timeframe is not specified in the rubric.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Working with Generated Rubric Suggestions & Evals

Auto-generated rubrics and evals are plausible but often have subtle issues, and they may not be complete.

Always use your own judgement and ensure that your rubrics are complete and accurate.

The Rubric **needs to be structured in a very specific way**, or the model might learn the wrong things from the rubric.

Consider rubric suggestions and evals as valuable tools, but always decide for yourself.

Rubrics are highly prompt-specific, but...

...the goal is a set of criteria that are **mutually exclusive** (they shouldn't overlap) and **collectively exhaustive** (taken together, they should fully define the ideal response).

MECE can be achieved by following **three principles:**

Atomicity

Each criterion should cover **exactly one** aspect

Specificity

Criteria should **precisely** state what is expected

Self-containment

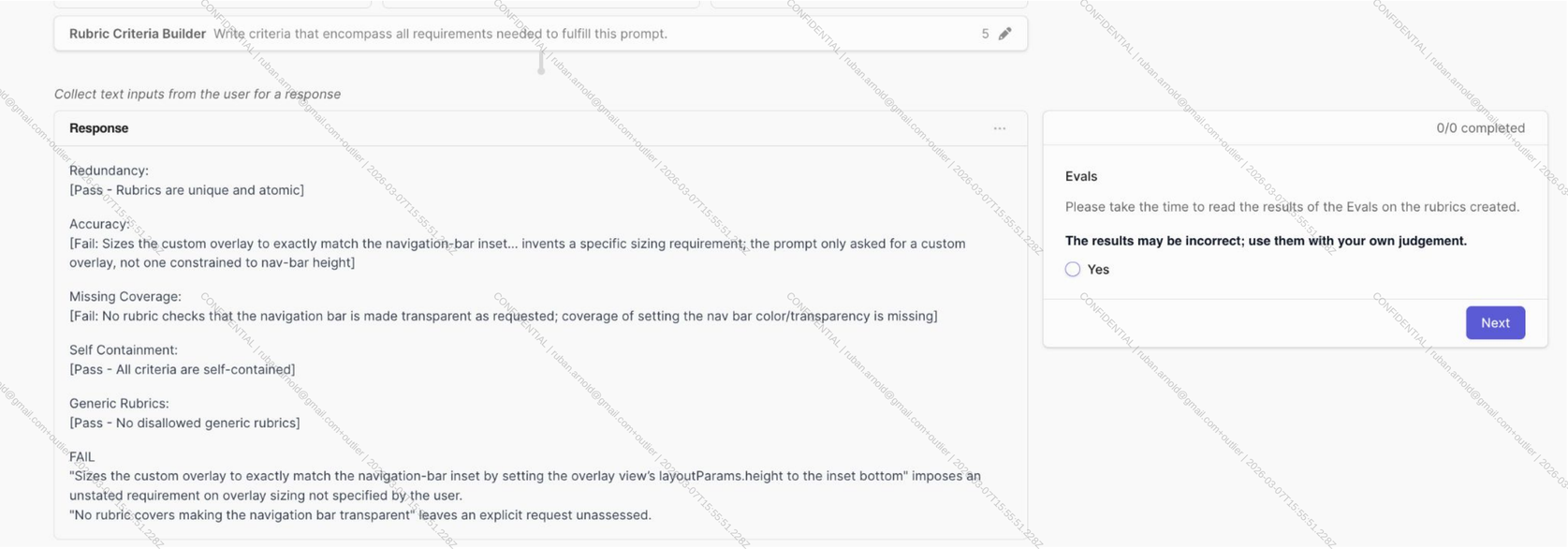
Each criterion should contain **all the information** needed to evaluate a response.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Working with Evals

Use them as a tool, but always use your own judgement.

Rubric evals are a **helpful tool** when crafting rubric criteria, but the are not an absolute source of truth. Read and consider these suggestions, and make edits to your rubrics based on them when they offer valid feedback.



CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Scoring Errors

Failing Issue

Overall Score Major Issue

- **[Fail - Overall Score Major Issue]**: The contributor scored a response with a 1/2 when objectively a 3/4/5 or vice versa.

Cascade Effect

Scoring errors can impact other areas of the task.

Reviewers should take care to check the selected response and the golden response. Both of these are likely to need adjustments when a major scoring error was made in the initial attempt.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Atomicity

Failing Issue

Atomicity of Rubric Criteria

[Fail - Criteria Atomicity]: Criterion groups unrelated constraints. Results in a rubric item with no clear focus.

It should read as a single coherent instruction, not a dump of requirements.

Example

Non-Atomic Criterion:

❌ "The function wraps the API call in a try/catch block to handle network errors and logs the error message to an external monitoring service like Sentry.

Atomic Criterion:

✅ The response includes a try/catch block around the API request logic to handle network errors.

✅ The catch block invokes a logging function or service (e.g., console.error, a custom logger, an external service like Sentry) to record error details.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Missing Criteria

Failing Issue

Missing Rubric Criteria

[Fail - Missing Criteria]
Criteria that are objectively 100% essential to meet the fundamental requirements of the prompt are missing, preventing the response from successfully fulfilling the core task if not included.

Example

Prompt:
*Write a Python function using the requests library to fetch **all** user records from the /api/users endpoint, handling **pagination** via the next_page_url field in the JSON response.*

✕ No criterion requires the code to **loop or recurse**, so code that returns only the first page of users could potentially pass the rubrics while failing to fulfill the core request.

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.228Z

Formatting

Failing Issue

Formatting of Rubric Criteria

[Fail - Multiple Formatting Issues]: 2+ criteria have one of the following formatting issues:

- Written in past tense
- Written as a question
- Starts with "The model"

Example

Bad Rubric Criteria Formatting:

❌ "The model used a context manager (e.g., with open(...) or Path(...).open()) for all file I/O operations."

❌ "Did the response invoke a context manager (e.g., with open(...) or Path(...).open()) for file I/O operations?"

Correctly Formatted Criterion:

✅ "The script utilizes a context manager (e.g., with open(...) or Path(...).open()) for all file I/O operations."

CONFIDENTIAL | ruban.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Rubric Category definitions

Code

→ Evaluates the **correctness, safety,** and **intent** of the code

- Focus on intent, not exact syntax
- Avoid requiring specific methods, libraries, or line-by-line implementations unless explicitly needed
- Accept alternative valid approaches if they meet the goal

Instruction Following (IF)

→ Ensures the model correctly follows the user's **explicit and implicit instructions**

- Check that all parts of the prompt are fully addressed
- No additions, omissions, or scope drift
- No reinterpreting or ignoring user requirements

Instruction Following (IF) rubrics must be mandatory.

Quality of Life (QoL)

→ Evaluates usefulness and clarity across three sub-dimensions:

- **Helpfulness** – relevance, usefulness, guidance, and solving the user's need
- **Style & Clarity** – readable code, clear structure, logical flow, visual organization
- **Verbosity** – concise yet complete; no rambling, no missing essential information

Quality of Life Rubrics (QoL) must be Nice-to-have.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Criteria Weights: Mandatory vs. Nice to Have

Failing Issue

[Fail - Criteria Weights Major Issue]:
>20% of the criteria weights are objectively wrong (Mandatory vs Nice to Have).

Mandatory

- Covers **explicit** or **implicit** requirements from the prompt.
- Represents something the response **must include** to be an acceptable response.

Nice to Have

- Improves response quality but is **not required** to fulfill the task.
- Reflects enhancements that **objectively** improve usefulness or clarity.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282

Vague or Irrelevant Criteria & Rewrite Issues

Vague / Generic

[Fail - Vague Criteria]

The criterion is too vague or abstract to be meaningfully evaluated, making it impossible to determine whether it has been met.

[Fail - Generic Criteria]

Non-QoL criterion is too generic (could be applied to virtually any prompt).

Irrelevant Rubric

[Fail- Irrelevant Criteria]

Criterion defined does not make the response objectively better OR it makes the response objectively worse.

Code Rewrite

[Fail - Code Rewrite Requirements]

The rewrite does NOT pass all the criteria.

If a rewrite fails any rubric, the task fails and the highest quality score it can receive is 2.

CONFIDENTIAL | ruben.arnold@gmail.com+outlier | 2026-03-07T15:55:51.2282